

Divide y vencerás

Dividir

Conquistar

Combinar

Búsqueda binaria $\leftarrow$ Arrays ordenados (verificar si existe ...)

Iterativo $\Theta(N)$

Divide y vencerás $\Theta(\log N)$

Máximo subArray $\leftarrow$ Array desordenado (casos consecutivos)

Iterativo $\Theta(N^2)$

Divide y vencerás $\Theta(N \cdot \log(N)) \leftarrow$ Necesario recombinar y comprobar el centro MaxSubArray Cruzada

No mejora la complejidad del algoritmo inmediato

Máximo de un vector

Transponer Matriz

Basados en divide y vencerás

QuickSort $\leftarrow$ Ordenar por pivote al final

Mejor $\Theta(N \cdot \log(N))$

Peor $\Theta(N^2)$

MergeSort $\leftarrow$ División por la mitad y uso de estructura adicional
aux array

Mejor $\Theta(N \cdot \log(N))$

Peor $\Theta(N \cdot \log(N))$

QuickSelect

Mejor $\Theta(N)$

Peor $\Theta(N^2)$

Karatsuba $\leftarrow$ Multiplicar números grandes

Teorema maestro

$$T(n) = p \cdot T\left(\frac{n}{q}\right) + f(n)$$

$$p \geq 1$$

$$q > 1$$

$$\left. \begin{array}{l} f(n) \in O(n^a) \\ a < \log_q(p) \end{array} \right\} \rightarrow T(n) \in \Theta(n^{\log_q(p)})$$

$$\left. \begin{array}{l} f(n) \in \Theta(n^a) \\ a = \log_q(p) \end{array} \right\} \rightarrow T(n) \in \Theta(n^{\log_q(p)} \cdot \ln n)$$

$$\left. \begin{array}{l} f(n) \in \Omega(n^a) \\ a > \log_q(p) \end{array} \right\} \rightarrow T(n) \in \Theta(f(n))$$

$$\exists c < 1 \forall n > n_0$$

$$p \cdot f(n/q) \leq c \cdot f(n)$$

Backtracking

Arbol de estados binario

- Estructura datos solución: $(x_1, x_2, \dots, x_n)$ donde $x_i = \{0, 1\}$
- Nodo del arbol: trata el elemento de la posición i del vector
- Parámetros estado: vector solución y nivel
- Llamada inicial: subconjunto SumaBack(vector, num, solución, nivel: 0, éxito)
- Solución: si $\text{nivel} == \text{vector.length} \rightarrow$ si suma números seleccionados $== \text{num}$
- Candidatos: no añadir / añadir el elemento i -ésimo del vector.
- Aceptable: si candidato $==$ añadir $\rightarrow$ si añadir $\text{vector}[i]$ a la solución no supera valor objetivo

siempre son inversos {

- Anotar: $\text{solución}[\text{nivel}] = 1$ (si candidato $==$ añadir); $\text{nivel} = \text{nivel} + 1$;
- Desanotar: $\text{nivel} = \text{nivel} - 1$; $\text{solución}[\text{nivel}] = 0$;

Arbol de estados combinatorio

- Estructura datos solución: $(x_1, x_2, \dots, x_n)$ donde $x_i = \{0, 1\}$
- Nodo del arbol: intenta añadir un único número del vector a la solución.
- Parámetros estado: vector solución, suma acumulada e id siguiente candidato
- Llamada inicial: subconjunto SumaBack3(vector, num, solución, id: 0, suma: 0, éxito)
- Solución: si $\text{suma} == \text{num}$
- Candidatos: $\text{id} .. \text{vector.length} - 1$
- Aceptable: $(\text{suma} + \text{vector}[\text{candidato}]) \leq \text{num}$
- Anotar: $\text{solución}[\text{candidato}] = 1$
- Desanotar: $\text{solución}[\text{candidato}] = 0$

```

void backtracking (Estado estado, Boolean éxito, ...) {
    if (esSolucion(estado)) {
        éxito.setValor(true);
    } else {
        inicializar_conjunto_de_candidatos();
        while (!éxito.getValor() && (hay_mas_candidatos())) {
            seleccionar_un_candidato();
            if (aceptable(candidato)) {
                próximoEstado = anotar_candidato();
                backtracking (próximoEstado, éxito, ...);
                if (!éxito.getValor()) {
                    desanotar_candidato();
                }
            }
        }
    }
}

```

Objeto para mantener la modificación y no perder el valor en las llamadas recursivas.

Estructura base para resolver problemas de backtracking

1 Solución ← Asignación if (nivel == N)
 2 Nodo ↑ index como parámetro

3 Estado Actual

4 Llamado Inicial

5 Solución Parcial

6 Candidato

7 Aceptable

8 Anotar

9 Desanotar

En selección óptima aquí es donde se hace la asignación de lo mejor encontrado.

Decisión de NO tomar rama

Actualiza nivel pero no aporta más

Hacer poda antes de tomar o no tomar las ramas.

Modificaciones sobre el esquema general de backtracking

1. Inclusión de estructuras de datos para albergar solución actual y la mejor solución hasta el momento.
2. Se generan todas las posibles soluciones (no se para ante la primera).
3. Siempre que se realiza la acción anotar se realiza desanotar.
4. Al encontrar solución se compara con la mejor solución hasta el momento.
5. Existe la posibilidad de poda para evitar recorrer estados que no conducen a una solución mejor.

Selección óptima

```
void seleccionOptima (Estado estado, Solucion solucion, Valor actual,  
                     Solucion mejorSolucion, Valor mejor, ...) {
```

```
    if (esSolucion(solucion)) {  
        if (actual ES_MEJOR mejor) {  
            mejorSolucion = solucion;  
            mejor = actual;  
        }  
    }
```

```
    } else if (hay_mas_candidatos) {
```

```
        for (candidatos) {
```

```
            if (acceptable(candidato)) {
```

```
                proximoEstado = anotar_candidato();
```

```
                seleccionOptima (proximoEstado, solucion, actual,  
                                mejorSolucion, mejor, ...);
```

```
                desanotar_candidato();  
            }  
        }
```

sin
poda

↓ con poda

```
        if (actual ES_MEJOR mejor) {
```

```
            seleccionOptima (proximoEstado, solucion, actual,  
                             mejorSolucion, mejor, ...);  
        }
```

Voraces

El algoritmo realiza n elecciones para un problema de tamaño n (ordenes habituales $\begin{cases} \theta(n) \\ \theta(n \ln(n)) \\ \theta(n^2) \end{cases}$)

$\theta(n * \theta(\text{elegir}(n)))$ $\text{elegir}(n)$ es una función que permite elegir un elemento entre los n objetos.

Suele dar buen resultado si la solución tiene estas propiedades:

• Propiedad voraz

Se confía en que tomando un valor óptimo local en cada paso, se llegue a una solución óptima global.

• Subestructura óptima

Cuando la solución óptima al problema contiene soluciones óptimas en los subproblemas.

El esquema voraz se basa en formar la solución a cada paso:

- Dado un conjunto finito de entradas C , un algoritmo voraz devuelve un subconjunto S (seleccionados) tal que $S \subset C$.
- Cada conjunto S que satisfaga las restricciones se le denomina prometedora, y si además minimiza o maximiza (según corresponda) S será una solución óptima.

- Conjunto C de candidatos

- Conjunto S de seleccionados

- Función solución $\leftarrow$ Comprueba si el conjunto seleccionado forma una solución.

- Función de validez o factibilidad $\leftarrow$ si el elemento más prometedora puede formar parte de la solución.

- Función de selección $\leftarrow$ ¿Cuál es el elemento que completa la solución?

‡ No puede haber sido rechazado o escogido con anterioridad.

- Función objetivo $\leftarrow$ Para la implementación de la función de selección.

‡ Intenta optimizar la solución.

Esquema general

```
public ArrayList<Candidato> voraz (ArrayList<Candidato> candidatos) {  
    inicializarSolucion(solucion);  
    while (! esSolucion(solucion) && quedanCandidatos(candidatos)) {  
        candidato = seleccionarCandidato(candidato);  
        eliminarCandidato(candidato, candidatos);  
        if (esValido(candidato, solucion)) {  
            añadirCandidato(candidato, solucion);  
        }  
    }  
    if (esSolucion(solucion)) {  
        return solucion;  
    } else {  
        return null;  
    }  
}
```

seleccionados
solucion

candidatos

Recorre en profundidad

1. Determinar **estructuras** para **ConjuntoCandidatos** y **ConjuntoSolución**, cómo **añadir/eliminar*** los **candidatos**.
2. Estudiar y definir la función **seleccionarCandidato**.
↳ **selección voraz**
3. Determinar función **esValido**.
4. Determinar **esSolucion** y **quedanCandidatos**

*.remove(new Object(value))

para eliminar
ese objeto

→ .remove(value)

elimina lo que sea que haya en
esa posición.

Esquema resumido para memorizar

IniSolucion ← inicializarSolucion
Quedan ← quedanCandidatos
Seleccionar ← seleccionarCandidato
Eliminar ← eliminarCandidato
Factible ← esValido
Añadir ← añadirCandidato
Solucion ← esSolucion

```
IniSolucion;  
while(!Solucion && Quedan){  
    candidato = Seleccionar;  
    Eliminar;  
    if(Factible){  
        Añadir;  
    }  
}  
if(Solucion){  
    return solucion;  
} else {  
    return null;  
}
```

Recordatorio métodos ArrayList

ArrayList

int size()
boolean isEmpty()
E get(int index)
boolean remove(Object o)
E remove(int index)
boolean contains(Object o)
int indexOf(Object o)
void add(E e)
boolean add(int index, E e) ← Desplaza
hacia la derecha
toArray()
Iterator